Chinese Remainder Theorem (for Gears)

I found a lot of discussion about how the Chinese Remainder Theorem can be used on encoders to determine the absolute position of a turret gear. But, most of the discussions just point to the Wikipedia page. Wikipedia pages about math are rarely written at a level for high school students, and there's not much discussion about the specific application to gears and encoders. I probably wouldn't understand it myself without a reference by Team SCREAM 4522, but even then it was a mystery where some of the numbers they were using came from (I had to consult the CAD). I hope to make it easier to understand so that teams can follow it step-by-step. While I'll use the term "turret gear", note that this has been successfully used for gears on elevators and other situations where you might want to know where your position is after a loss of power. I'll talk primarily about using two encoders.

Physical requirements

Gears

The choice of encoder gears matters for being able to successfully apply the Chinese Remainder Theorem and for the number of times your turret gear can turn before you lose track of its *absolute* position. We'll refer to these as variables "e1_teeth" and "e2_teeth", and we're deciding on the number of teeth for each gear. When referring to the turret number of teeth, we'll just use "t_teeth".

The first requirement is that the number of teeth on the two encoder gears are co-prime. An easy way to ensure this without having to compute the greatest common factor, is pick one that is known to be prime, and select a *different* number that is less than or equal to that prime number plus one.

The second requirement is that (e1_teeth*e2_teeth)/t_teeth is greater than or equal to the number of rotations you need to be able to track. e1_teeth*e2_teeth is a reset point, where the pattern of remainders will start to repeat. Beyond that point, we'll lose track of the total number of rotations. If e1_teeth*e2_teeth is an integer multiple of t_teeth, we'll still be able to know the relative position of the turret, but not the absolute number of rotations. So, if t_teeth is 47 and we need to be able to at least know when it has gone 3 rotations, e1_teeth*e2_teeth will need to be greater than 141 (47*3).

Third, the gears need to move tooth-per-tooth with the turret gear, whether directly connected to it or driven by an intermediate gear.

Encoders

The encoders will need to be absolute single-turn rotary encoders. Absolute, meaning that we do not lose track of the position with loss of power. Single-turn means it keeps track of

the current angle but not the total number of rotations. For many cases, a multi-turn absolute encoder would eliminate the need for this solution. The encoder may be able to receive a command to be zeroed, but if not, when we physically zero the turret (that is, put it at one physical extreme of its clockwise or counterclockwise motion), we'll need to record the offsets for each. In these cases, all of the math done below will involve subtracting the offset from the raw value so that our zero can be in a meaningful place for our robot. We'll refer to the corrected values of the encoders and turret with e1_val, e1_val, and t_val.

The Math (the fun part)

For the purposes of this explanation, I'm going to set teeth values for e1, e2, and t that satisfy the criteria above.

t_teeth = 99
e1_teeth = 49
e2_teeth = 67

Note that this means if the turret gear rotates beyond 3283 teeth (over 33 rotations), we'll lose track of its position and need to re-zero. If you're working with a turret that uses a cable chain or wire loom and not a slip ring, that's probably much more than enough.

Get a reading from the encoders

I don't have a physical robot sitting in front of me that I can get readings from, so we'll have to simulate the encoder reading by picking a number between 0 and 11880 degrees that t_val might be for zero to 33 rotations. For this example, we'll use 1055.7. While the teeth move one-to-one, the encoders cycle at different rates, with a smaller gear making more rotations. We'll use the gear ratios to find our simulated e1_val and e2_val.

e1_val = 1055.7 * (99/49) % 360 = 332.9449
e2_val = 1055.7 * (99/67) % 360 = 119.9149

Note that these values represent the number of degrees. Modulo 360 is used because our single turn encoders cannot track the total number of rotations, even though we will be determining the total number of rotations.

Calculate the rotations

We're going to plug these in the formula from Team SCREAM, but first let's break apart the calculation so we can see what it's doing.

Here's the formula: (n + (E/360))Gr = A

"n" is an array of all possible countable *rotations*. Now … in their example, they used the turret gear and started the array with 1. But, remember how we said you can only make a certain number of rotations before you repeat? We can use that to reduce the size of the array. So, for each encoder, you only need an array from 0 to one less than the teeth count of the other gear.

E is the encoder in degrees, so we divide it by 360 to put it into rotations, so that is in like terms with n. We add these together, and multiply by the gear ratio.But let's take the gear ratio one part at a time. When we multiply by e1_teeth, it's more precise to call this the teeth *per rotation*. So, before we divide by our t_teeth we've counted the number of *teeth* that have rotated. Dividing by t_teeth per rotation gives us turret rotations. The first few rows in Excel where I did the calculation reveal the results and our answer is 2.9325 rotations.

n + e1/360	A1	n + e2/360	A2	
0.924847	0.457753	0.333097	0.225429	No Match
1.924847	0.952702	1.333097	0.902197	No Match
2.924847	1.447652	2.333097	1.578965	No Match
3.924847	1.942601	3.333097	2.255732	No Match
4.924847	2.437551	4.333097	2.9325	Match
5.924847	2.9325	5.333097	3.609268	No Match

But … where is the remainder used?

So, we said we were going to use the Chinese Remainder Theorem, but you might notice … we never took the remainder using a modulo with the carefully chosen co-prime number of teeth. How is this the Chinese Remainder Theorem at all? Let's rewind to the (n + (e1_val/360) * e1_teeth part of the equation, and let's distribute the e1_teeth through first. Now we have (n*e1_teeth) + (e1_val/360) * e1_teeth. So, remember the e1_val/360 is a partial rotation in teeth. It's the result if we were to perform a modulo operation on the total number of rotations (the left term is always going to be a multiple of e1_teeth, so it disappears with modulo). So, why didn't we have to do modulo for the number of teeth? The single-turn encoder kind of already did that for us.

And what angle are we aiming at now?

For cable chain management, the number of rotations absolutely matters. We don't want to drive past our limit and rotate too far. But we're using that turret to aim at something. So, where are we aiming? Take the decimal portion of the rotation and multiply it by 360, and we get .9325 * 360 = 335.7 degrees (It's all in good fun COMETS)!